

Bachelorarbeit

**A Memory-Safe Integration of Remote
Attestation with TLS**

Arismunandar

11. August 2025

Technische Universität Dresden
Fakultät Informatik
Institut für Systemarchitektur
Professur für Betriebssysteme

Betreuender Hochschullehrer: Prof. Dr.-Ing. Horst Schirmeier
Betreuender Mitarbeiter: Dr.-Ing. Carsten Weinhold

Selbstständigkeitserklärung

Hiermit erkläre ich, dass ich diese Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Dresden, den 11. August 2025

A handwritten signature in black ink, consisting of stylized, cursive letters that appear to be 'A' and 'S'.

Arismunandar

Abstract

Remote attestation ensures the integrity of software and hardware running on remote systems. This helps overcome the inability of Transport Layer Security (TLS) to confirm the reliability of endpoint software stacks. The "intra-handshake" technique, in which attestation data is incorporated into TLS handshake extensions without the need for protocol changes or extra network round trips, has been used in recent studies to successfully integrate remote attestation with TLS. RATLS by Walther et al. is a notable implementation that includes attestation data in TLS handshake messages by extending OpenSSL using its callback-based extension API. However, OpenSSL is a C-based library and thus prone to memory safety vulnerabilities. In order to overcome this limitation, this work uses Rust and the rustls library to implement a memory-safe version of remote attestation integrated with TLS. The implementation makes use of Rust's ownership system and type safety while maintaining the accepted intra-handshake methodology. Support for various hardware roots of trust is made possible by a modular architecture that uses Rust's trait system to isolate platform-specific attestation providers from generic protocol logic. Comparing the implementation to standard TLS, evaluation shows that the remote attestation extension introduces a significant performance overhead in exchange for stronger security properties.

Contents

List of Figures	V
List of Tables	1
1 Introduction	2
1.1 Trust in Distributed Systems	2
1.2 State of Art	2
1.3 Motivation and Objectives	3
1.4 Rust for Cryptographic Systems	3
2 Background and Related Works	5
2.1 Overview of TLS	5
2.2 Remote Attestation	6
2.3 Foundational Libraries	7
2.4 Existing Remote Attested TLS Approaches	8
2.5 Importance of Memory Safeness	9
3 Design	12
3.1 Core Design Principles	12
3.2 Security Requirements	13
3.3 System Architecture	14
3.4 Memory Safety Considerations	16
4 Implementation	18
4.1 Communication Flow	18
4.2 Core Attestation Module	19
4.3 Client-Side Integration	26
4.4 Server-Side Integration	31
4.5 TPM Implementation	33
5 Evaluation	37
5.1 Performance	37
5.2 Security	41
6 Future Work	43
7 Conclusion And Outlook	44
Bibliography	45

List of Figures

1	Short description	14
1	Short description	18

List of Tables

1	TLS Handshake Performance Comparison	38
2	TLS + Server TPM Attestation Handshake Latency Breakdown	38
3	TLS + Mutual TPM Attestation Handshake Latency Breakdown	38

1 Introduction

1.1 Trust in Distributed Systems

Even in the digital ecosystem of today, trust between systems that are remote from each other is still arguably the biggest hurdle in cybersecurity. Transport Layer Security (TLS) has been the foundational protocol for securing communications over networks to provide guarantees for confidentiality, integrity, and authenticity [Res18]. TLS is however limited in ways that can be problematic in high-security environments and in zero-trust designs.

TLS does authenticate identities of endpoints using a Public Key Infrastructure (PKI), but it does not help determine what software and hardware configurations the endpoints are using. For example, attackers that compromised a system and acquired private keys could impersonate an endpoint using valid TLS certificates from a complete untrustworthy environment. The identity validation of endpoint authentication can be disconnected from a trustworthy platform, which led to the development of remote attestation technology.

By enabling systems to cryptographically demonstrate the hardware and software integrity to remote verifiers, remote attestation seeks to overcome this restriction. Hardware root of trusts such as Trusted Platform Modules (TPMs) or secure enclaves are leveraged in remote attestation to provide verifiable measurements regarding the system's state, including its boot sequence, operating system configuration, and running applications[Con24].

1.2 State of Art

Recent work has shown that remote attestation can be combined with TLS using a process called "intra-handshake" wherein attestation data is placed in handshake extensions within TLS. There are now a number of implementations, such as TLS+RA [Wei+25], that utilize this functionality to incorporate remote attestation into a TLS connection where existing attestation protocols do not need to be run separately and we do not need to introduce additional network round trips to allow for attestation, thus allowing for an efficient improvement to our current TLS infrastructure.

While TLS-integrated remote attestation is promising, the current implementations have a major problem: they are still implemented on C-based cryptographic libraries like OpenSSL which are prone to memory safety issues. The history of vulnerabilities in cryptography shows that memory safety issues can completely negate the assurances

of even the most trusted protocols. The Heartbleed vulnerability (CVE-2014-0160) [heartbleed2014] highlights that one buffer over-read in OpenSSL could lead to eliminating the required level of security by making private keys, session keys, and user data public, and thereby turning the supposed protections worthless.

These shortcomings from C-based implementation represents a major roadblock to the feasibility of deploying these types of systems into high-assurance domains.

1.3 Motivation and Objectives

This work fills a gap between the improved security properties of remote attestation and the memory safety properties of cryptographic systems. While former work has established that remote attested TLS can provide better security benefits than standard TLS, and while it has plenty of other beneficial applications, the reliance on memory-unsafe implementations has limited the real-world adoption in environments that are supposed to provide better security guarantees.

The primary goal of this work is to demonstrate that memory-safe remote attestation can be used in combination with TLS without sacrificing the security properties or protocol compatibility established in previous research. Rust's combination of memory safety guarantees with zero-cost abstractions is leveraged in this work with the hope of taking the security benefits of remote attestation while removing memory safety flaws from previous implementations.

1.4 Rust for Cryptographic Systems

The selection of Rust as the implementation language for this work is not just a subjective choice, but rather a purposeful choice based on best practices that resolve some of the fundamental issues with the construction of cryptographic systems. Rust provides zero-cost abstractions while guaranteeing memory safety and therefore ideal for high-assurance tasks as it provides correctness.

Memory safety in cryptographic implementations usually relied on the use of defensive programming practices, extensive testing, and runtime checks to prevent memory violations, which is reactive because it tries to check for and remediate memory violations after they have already occurred within the source code. Moreover, given the complex nature of cryptographic protocols, executing all possible execution paths is painstakingly difficult when memory safety is considered through solely traditional testing methods, and without automated memory checks.

The ownership system of Rust transitions memory safety assurances from runtime to compile time. Rust's borrow checker enforces clear rules for access and mutation privilege for their memory [The24]. This additional level of protection is what makes Rust valuable for cryptographic implementations.

The Rustls library is an example of using Rust in practice. OpenSSL and other libraries based on it have long history of memory safety vulnerabilities. Rustls has used Rust's enforcement of memory safety in development to explicitly mitigate the risk of software vulnerabilities. All of the security issues discovered in Rustls were mostly logic errors or denial-of-service issues, which are not memory corruption issues [Cur20].

2 Background and Related Works

2.1 Overview of TLS

TLS is a security protocol that is developed to ensure secure communication between computer networks. Nowadays it is used to secure communications across the internet such as web browsing, email messaging, and app-to-app communication [Clo24].

2.1.1 Handshake Process

The TLS handshake is series of steps to establish a TLS connection [Res18]. The handshake begins to form a secure communications link when a client device connects to a TLS-enabled server. The handshake achieves important security goals that include version and cipher suite negotiation, mutual authentication of the communicating parties, and the creation of session keys for symmetric encryption.

Initial Agreement The client start off by sending a *Client Hello* message which contains a random number (nonce), available TLS versions, list of cipher suites, available compression methods, and the extensions. The server then replies with a *Server Hello* message containing the server nonce, selected TLS version, cipher suite, compression method, and session id.

Authentication The server's authentication to the client is done by the server sending a digital certificate to the client which contains the public key and is signed by a known Certificate Authority (CA). The client authenticates the certificate by authenticating the CA signature, checking that the certificate is valid with respect to expiration and confirming that the server demonstrates it has the corresponding private key. The server demonstrates it has the private key by signing all of the handshake messages exchanged thus far with its private key, which allows the client to verify the signature with the public key in the certificate. In a mutual TLS session, the server will also require the client to authenticate in a *Certificate Request* message and the client will respond with its own certificate and a *Certificate Verify* message which will be a digital signature created with its private key.

Key Exchange The client and server will then do a key exchange protocol with the selected cipher suite to generate the shared secret that will be used to derive the symmetric encryption keys.

Completion As the final step both sides send finished messages with a hash of all handshake messages to cryptographically prove the handshake has not been altered.

Limitation The strong cryptographic protection offered by TLS faces security constraints which led to remote attestation development:

PKI Dependency TLS depends on certificate authorities to authenticate endpoint identities through the Public Key Infrastructure (PKI). The trust model built on certificate authorities makes all connections vulnerable when any CA experiences security breaches.

Insufficient Assurance Through TLS certificates endpoints can verify identity but system hardware and software configurations remain unknown to the authentication process. If the private key were to be leaked an attacker could act like a legitimate party.

2.2 Remote Attestation

Remote attestation functions as a security mechanism which permits one system (verifier) to verify the trustworthiness of another system (attester) through cryptographic evidence of its hardware and software setup. The solution solves the basic issue of trust establishment in distributed systems when direct physical inspection and access are not possible.

Root of Trust A Root of Trust (RoT) is a collection of hardware or firmware components that are trusted to execute security-essential functions and cannot be altered without detection [Uti24]. The RoT is the basis for establishing trust in a system, because it securely measures other components in the system during the boot process and signs those measurements to enable remote verification. A common example of a RoT is Trusted Platform Module (TPM) [Tru19]. The TPM contains cryptographic keys, generates secure measurements, and protects its measurements from tampering.

Measurement and Reporting Remote attestation measures system software and configuration by calculating cryptographic hashes of essential system components. The attestation measurements are compiled into reports which remote parties can verify.

Verification The verifier obtains the attestation report to check both the cryptographic integrity of the report and the system measurements against predefined trusted system configurations. Once received the report will be validated against known good values, checking both the cryptographic authenticity of the report and whether the reported measurements correspond to trusted system configurations [Ede25].

2.3 Foundational Libraries

The two main libraries used for this work are Rustls (for the TLS implementation) and TSS-ESAPI (for the TPM implementation).

2.3.1 Rustls

Rustls is a modern TLS library developed in Rust that offers a memory-safe alternative to established TLS implementations in C like OpenSSL [Rus24]. The library was designed from the bottom up to take advantage of Rust’s system of ownership and guarantees of type safety while ensuring high performance for TLS operations.

Rustls is built on a principle of security first and prioritizes correct and auditable implementation over support for legacy features. Rustls does not implement insecure or deprecated features of TLS and only provides support for the TLS 1.2 and TLS 1.3 protocols with strong cipher suites.

Rustls implements a modular architecture where protocol logic is separated from cryptographic primitives and certificate handling. It uses trait based abstractions that define interfaces for cryptographic operations, certificate verification, and protocol extensions. This allows users to add functionality cleanly without sacrificing the security properties of the underlying system.

Even with its focus on safety, rustls not only performs similarly to optimized C implementations but it can also do this without the overhead of garbage collection while still maintaining memory safety, making it suitable for high-performance systems.

2.3.2 TSS-ESAPI

The TSS Enhanced System API (ESAPI) is a high-level Rust interface for accessing Trusted Platform Module (TPM) 2.0 devices [Par24]. The ESAPI is part of the TCG Software Stack (TSS) 2.0 specification, and is the primary interface layer for applications needing TPM functionality.

The ESAPI library provides a complete set of functions that map onto TPM 2.0 commands, which removes the complexities of working directly with a TPM. The library takes care of session management, authorization mechanisms, parameter marshaling/unmarshaling, etc. This removes the need for an application to understand the structure of TPM commands.

The library supports interactions with both hardware and software implementations of TPM, letting it be used in development, testing, production, and other deployment environments.

2.4 Existing Remote Attested TLS Approaches

The key challenges of cryptographic binding, performance, and deployment compatibility have been addressed by a number of research initiatives that have contributed various strategies for integrating remote attestation with TLS.

2.4.1 RATLS

One of the earliest complete implementations of remote attested TLS that uses the intra-handshake technique is RATLS [WWR22]. The system incorporates attestation information directly into TLS handshake messages by building upon OpenSSL’s extension mechanism.

Protocol Design The three-phase RATLS process supports mutual attestation and is similar to the conventional TLS handshake. Both the client and the server include attestation request extensions in their ClientHello and ServerHello messages during the request phase. These extensions contain a randomly generated nonce that acts as a challenge for the peer’s attestation. Both parties obtain quotes from their attestation providers during the attestation phase, which contain platform measurements, the received nonce, and their individual TLS public keys. Both the client’s and the server’s certificate messages contain these quotations as extensions. In order to ensure that nonces match, public keys are consistent between certificates and attestation reports, and platform measurements show reliable system states, both parties validate their peer’s TLS certificate and attestation report at the end of the verification phase. The TLS public key is included into the attestation report measurements to accomplish the cryptographic binding in RATLS. This stops relay attacks, in which attestation reports are sent from trustworthy systems to compromised endpoints, by guaranteeing that the system producing the attestation report has the private key matching the TLS certificate.

Implementation Architecture On top of OpenSSL’s extension API, RATLS implements a callback-based architecture. OpenSSL uses registered callback functions to add or parse extension information when processing handshake messages. In order to coordinate various handshake phases and keep track of attestation-related data, RATLS keeps extra state information in the SSL session context. Platform-specific attestation implementations and generic RATLS protocol logic are separated by the system’s plugin architecture for attestation providers. Through standardized callback interfaces that include features for creating requests, generating reports, verifying quotes, and sealing session secrets for session resumption, this design makes it possible to support various attestation technologies.

Session Resumption Support In order to preserve attestation guarantees across connection re-establishments, RATLS extends TLS session resumption. Through sealed storage supplied by the attestation hardware, the server creates client and server secrets

that are tied to the current platform state during the initial handshake. Both parties must show that they can unseal these secrets when a client tries to resume a session, demonstrating that their platform state is still in line with the configuration that was initially attested. This method maintains attestation-based security guarantees while facilitating effective reconnection between peers that communicate frequently. By detecting any unauthorized software modifications that may have taken place between connections, the sealed storage mechanism makes sure that session resumption is only feasible when both the client and the server are still in previously verified states.

2.4.2 TLS+RA

When it comes to TLS-attestation integration, TLS+RA presents the ideas of deployment independence and failure independence [Wei+25]. TLS+RA emphasizes keeping TLS certificate infrastructure and attestation mechanisms separate in order to provide additive security properties, even though it takes a similar intra-handshake approach to RATLS.

Double Linking Architecture TLS+RA’s key innovation is its double linking technique, which establishes two separate cryptographic bindings—one between the TLS session and the certificate (standard TLS behavior) and another between the TLS session and the attestation report. Despite using different signing keys and infrastructure, both links are based on the same transcript of the TLS handshake. The connection between the session and the certificate-based identity is established by the TLS link using the standard transcript hash signed with the TLS private key. The handshake transcript and the ephemeral Diffie-Hellman shared secret, both of which are signed by the attestation root of trust, are included in the linking hash used by the attestation link. Since the shared secret contains information that is only known by the actual communication endpoints, it is essential to include it in the attestation link. This prevents relay attacks even in the event that TLS private keys are compromised.

Independent Failure The design of TLS+RA allows for independent failure characteristics, meaning that all security guarantees are not entirely eliminated in the event that either security mechanism is compromised. The attestation link (linking hash) can still identify that the encrypted channel does not terminate in the anticipated attested environment even if the TLS private key is compromised (as in Heartbleed-type vulnerabilities). On the other hand, the TLS certificate can still guarantee administrative control over the endpoint even if the attestation root of trust is compromised. This feature offers real additive security as opposed merging the two authentication factors.

2.5 Importance of Memory Safeness

The correctness and security of cryptographic protocols’ implementations are just as important to their security as their theoretical design. Implementation safety is a

crucial factor for high-assurance systems since memory safety flaws in cryptographic implementations have frequently shown themselves to be capable of totally compromising protocol security [Cod14][21].

2.5.1 Memory Safety Vulnerabilities in C-Based Implementations

Historically, C-based cryptographic libraries, including OpenSSL, have documented history of memory safety vulnerabilities. These vulnerabilities have caused serious security issues in practice. The vulnerabilities include buffer overflows and buffer underflows, use-after-free, double-free, and null pointer dereference. Each category describes a different means by which the lack of memory management can lead to exploitable security vulnerabilities.

Memory safety vulnerabilities can severely compromise cryptographic security, as shown by the Heartbleed vulnerability [Cod14]. This flaw in OpenSSL’s TLS heartbeat extension, a buffer over-read vulnerability, allowed attackers to read arbitrary memory from server processes which in turn meant they could potentially access private keys, session keys, sensitive user data, etc.

Another example is CVE-2021-3449 [21], a NULL pointer dereference vulnerability in OpenSSL’s TLS v1.2 renegotiation mechanism. It was caused by sending maliciously constructed `ClientHello` messages that contained no `signature_algorithms` extension but did contain the `signature_algorithms_cert` extension. This could lead to a crash of the server, in turn resulting in denial of service.

A single memory safety vulnerability completely bypassed all of the security guarantees despite the fact that TLS provided strong cryptographic protection. The basic difficulties of manual memory management in security-critical code are highlighted by the frequent discovery of similar flaws in C-based cryptography implementations. Memory safety bugs are common and dangerous because of the complexity of cryptographic protocols, performance requirements, and the intrinsic insecurity of C memory management.

2.5.2 Ensuring Memory Safety

Memory safe programming languages typically have automatic memory management that mitigates entire classes of vulnerabilities analogous to C/C++ implementations [Cyb24]. Memory safe languages such as Rust and Go employ different approaches to memory safety, and each approach has pros and cons for cryptographic implementations. Rust may provide a more favorable memory safety guarantee because it can provide memory safe programming without the need for garbage collection and runtime overhead associated with garbage collection, making Rust a viable language for developing cryptographic implementations. Rust’s zero-cost abstractions assists in efficiently completing performance-critical cryptographic operations while also providing compile-time checks for memory safety violations, made possible by the ownership model, and borrowing rules.

Rustls is a full TLS 1.3 implementation that has been security audited [Cur20], and used in production systems, and serves as a practical example of how memory safe TLS implementations can be accomplished. Rustls is memory safe, and it can achieve performance similar to a lot of the C based implementations; as a result, Rustls provides an excellent opportunity for further additional developments on top of it.

3 Design

The memory-safe TLS-integrated remote attestation implementation’s design addresses the significant memory safety issues with current C-based implementations while sticking to the accepted guidelines from earlier research. The design makes use of Rust’s unique safety features while remaining compatible with tried-and-true protocol approaches.

3.1 Core Design Principles

Memory Safety First Ensuring memory safety throughout the attestation flow is the main design principle. In contrast to C-based implementations that depend on runtime checks and developer discipline, this design makes use of Rust’s type safety and ownership system to stop memory safety violations at compile time. This is an important aspect because memory vulnerabilities in cryptographic implementations have been shown to be able to bypass protocol security, as per vulnerabilities like Heartbleed. By moving these guarantees to compile time, the implementation gets rid of entire classes of vulnerabilities that could violate attestation integrity.

Protocol Compatibility The implementation retains cryptographic binding between TLS sessions and attestation reports while avoiding extra network round trips by embedding attestation information within TLS handshake extensions, following the well-established intra-handshake methodology shown by RATLS and TLS+RA. This design decision was made in order to remain compatible with other attestation protocols, such as Intel SGX or AMD SEV, while avoiding the additional expense, in complexity and performance overhead, of separate attestation handshakes. Intra-handshake was shown to be effective in past work [Wei+25] [WWR22] and seemed to be a reasonable option for remaining compatible with existing protocols while providing memory safety.

Type-Safe Abstractions The design uses Rust’s trait system to provide clean, type-safe abstractions for attestation providers instead of the callback-based methods found in C implementations. This guarantees that the memory safety of the core implementation cannot be compromised by platform-specific code. The driving force behind this decision was the need to prevent type confusion errors and guarantee that attestation operations adhere to strict type safety contracts. Rust uses traits to enforce compile-time type checking and barring types from being utilized improperly.

Modular Architecture By using defined trait interfaces to divide generic protocol logic from platform-specific implementations, the design allows for the support of a variety of attestation technologies while upholding strict safety contracts. This modularity allows

extensibility without losing safety by allowing new attestation providers to be added without altering the core implementation and using the trait system to make sure that all attestation providers acknowledge the same safety requirements.

3.2 Security Requirements

The same security standards set for earlier TLS-attestation integrations must be met by the implementation:

Maintain TLS Security Properties All the typical TLS security guarantees (confidentiality, integrity, or authentication) must be preserved while augmenting with attestation functionality.

Support Mutual Attestation Clients and servers should be able to attest to their software stacks to each other, which could be useful under zero-trust deployment scenarios.

Channel Binding Bind the attestation report to the TLS session using a *linking hash*. The *linking hash* is computed over the TLS handshake transcript, the DHE-derived shared secret, and the attestation nonce. By including the shared secret, only the two parties to the TLS session are able to compute the hash, and thus are protected from relay attacks. In addition to that, freshness is ensured by including the nonce. With this *linking hash*, the attestation process builds an independent cryptographic link between the TLS session and the attestation evidence, while complementing the existing binding of the TLS certificate. This also ensures that the secure channel terminates in the same trusted environment that was used to generate the attestation report [Wei+25].

Secure Failure To avoid downgrade attacks, Attestation failures should terminate the connection instead of reverting to unattested connections.

3.3 System Architecture

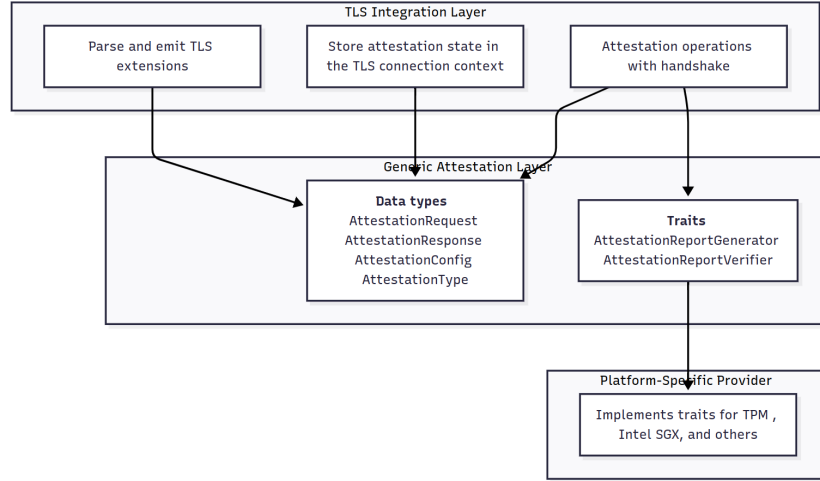


Figure 1: remote attested rustls layered diagram

A layered design is used in the system architecture to keep type safety across all interactions while separating concerns. The generic attestation layer, the platform-specific provider layer, and the TLS integration layer make up the three main layers of the architecture. This three-layer design was chosen over alternative designs for the following reasons: Each layer has their own responsibility, making the system easier to understand, test, and maintain. Furthermore, adopting the layered method will allow new attestation technologies to be added at the provider layer without impacting the TLS integration or the generic attestation logic.

3.3.1 TLS Integration Layer

The TLS integration layer has two responsibilities: creating attestation information contained in the TLS handshake extensions, and managing the relationship between attestation operations and the TLS protocol state. The attestation integration layer limits what it has to manage by providing support for, and managing the state of, the following:

- The processing of attestation related TLS extensions, including processing handshake messages that include attestation related TLS extensions.
- Coordinating the generation of attestation requests and responses with the progress of the TLS handshake within the context of the TLS handshake.
- Storing attestation state in the TLS connection context.

- Correctly sequencing the execution of attestation operations with respect to the phases of the TLS handshake.

To carry attestation relevant data in an ordinary TLS handshake message without adding or modifying the protocol, the implementation uses the standard way of forming extensions in TLS 1.3 and uses two custom extensions, `AttestationRequest` (0xff00) and `AttestationResponse` (0xff02). 0xff00 and 0xff02 both fall into the IANA TLS `ExtensionType` private use range. Using values from this range is safe for experimental and custom extensions to avoid collision with IANA registered extensions [Int25].

3.3.2 Generic Attestation Layer

Using Rust's trait system, the generic attestation layer offers platform-neutral abstractions for attestation operations. All platform-specific implementations must adhere to the fundamental attestation data structures and behavioral contracts specified in this layer.

Core Data Structures The layer specifies `AttestationRequest` and `AttestationResponse` structures that contain evidence and attestation challenges in a format that is independent of platform. The Codec trait for serialization in TLS extensions is implemented by these structures.

Abstractions Based on Traits The interface for attestation operations is defined by two main characteristics:

- Systems can produce attestation evidence in response to challenges with the `AttestationReportGenerator`.
- System verification of attestation evidence from distant peers is made possible by `AttestationReportVerifier`.

Configuration Management Attestation-related settings are managed by a configuration structures `AttestationConfig`, which also keep track of references to suitable provider implementations via trait objects.

3.3.3 Platform-Specific Provider Layer

For specific attestation technologies, the generic attestation traits are implemented by the platform-specific provider layer. While providing a consistent interface to the generic layer, each provider encapsulates the platform-specific methods of interacting with hardware roots of trust.

Attestation Technology Coverage Through the `AttestationType` enumeration, which contains predefined constants for TPM 2.0, Intel SGX, AMD SEV-SNP, and extensibility for custom implementations, the architecture supports a variety of attestation technologies.

Code Containment Platform-specific provider could invoke unsafe code (e.g. `tss-esapi` relies on a FFI wrapper, `tss-esapi-sys`). The model does not assert that memory safety is guaranteed in these providers. Instead, the model encapsulates unsafety behind trait boundaries and only allows systems to interact with these providers through type checked interfaces. All FFI calls will happen in the provider and the core and TLS layers only have access to the provider through safe trait methods that return `Result<_, Error>`. Aside from that the provider code must use bounded (un)marshalling and check lengths before use. Even if a code error occurs, the provider will forward the error, rather than panicking. Any misbehavior caused from an implementation error will lead to a verification failure, rather than undefined behavior.

3.4 Memory Safety Considerations

Through a number of strategies, the design specifically addresses memory safety issues that have impacted earlier implementations.

3.4.1 Ownership and Borrowing

Rust's ownership system is used by all attestation-related data structures to guarantee memory safety. In order to prevent use-after-free or double-free errors, attestation requests and responses use owned data (`Vec<u8>`) instead of raw pointers.

`Arc<dyn Trait>` is used by configuration structures to share ownership of trait objects, allowing safe sharing across thread boundaries and guaranteeing that attestation providers are valid for the duration of connections referencing them.

3.4.2 Type System Guarantees

By using Rust's type system, the type-based design applies certain constraints to attestation operations at compile time, through the trait system. The `AttestationReportGenerator` and `AttestationReportVerifier` traits will dictate the exact signature of functions, thus preventing mismatched data types between attestation requests and responses. If platform-specific implementations are missing any of the required methods, or provide incorrect types, they will not compile unless they implement all of the required trait methods and provide the correct parameter and return types. The design is intended to exclude missing features or out of order in-memory escapes that can cause runtime errors or memory corruption due to incorrect type references.

Aside from that the current rustls Codec trait system, which offers safe serialization and deserialization without the buffer overflow risks typical of C implementations, is used for extension parsing and generation.

3.4.3 Error Handling

All operations with the potential to fail return `Result<T, E>` types which require explicit error management. This was chosen for a few reasons: It prevents implicit error conditions that could possibly lead to undefined behavior, it makes error paths explicit and auditable, it upholds Rust's philosophy that failure conditions should be explicit types in the type system, and it allows operations to manage errors with better recovery options and debugging.

The design also allows for fail-safe defaults where if an attestation fails the connection is terminated, instead of falling back to unauthenticated connections. This conservative approach was selected to help prevent downgrade attacks and enable exposures from failed attestations to be leveraged to bypass security to connections that are not reliably authenticated.

4 Implementation

4.1 Communication Flow

The full handshake flow is depicted in Figure 1, which also shows the exact integration points where attestation operations are integrated into the baseline TLS 1.3 protocol.

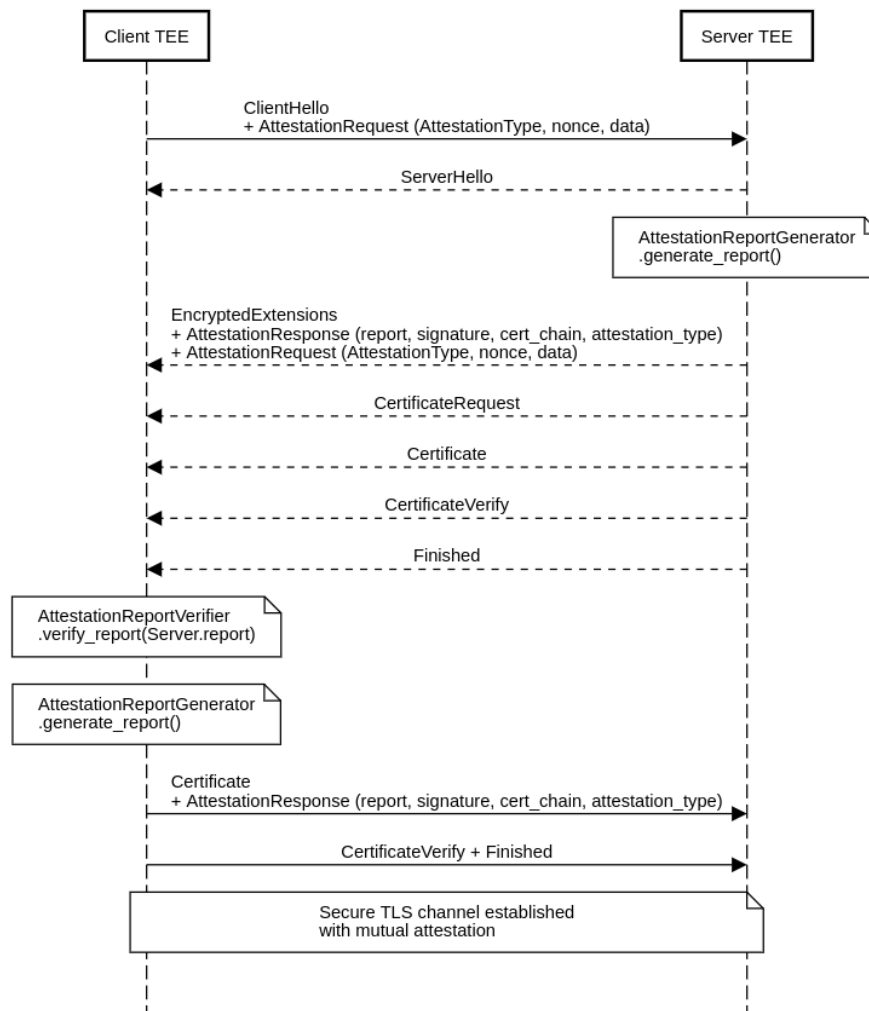


Figure 1: Sequence diagram of remote attested rustls

For the remote attestation integration, a separate attestation module that offers attestation operations is created. This module is intended to be used independently of particular attestation technologies since it is a self-contained part of Rustls.

4.2 Core Attestation Module

4.2.1 Type System

```

1      pub struct AttestationType(pub u32);
2
3      impl AttestationType {
4          pub const TPM: AttestationType = AttestationType(0x0001);
5          pub const SGX: AttestationType = AttestationType(0x0002);
6          pub const SEV: AttestationType = AttestationType(0x0003);
7          pub const CUSTOM_START: u32 = 0x0004;
8
9          pub const fn custom(id: u32) -> Self {
10             AttestationType(Self::CUSTOM_START + id)
11         }
12
13         pub fn description(&self) -> &'static str {
14             match self.0 {
15                 0x0001 => "Trusted Platform Module 2.0",
16                 0x0002 => "Intel Software Guard Extensions",
17                 0x0003 => "AMD Secure Encrypted Virtualization",
18                 _ if self.0 >= Self::CUSTOM_START => "Custom Attestation
19                     Technology",
20                 _ => "Unknown Attestation Technology",
21             }
22         }
23
24         pub fn short_name(&self) -> String {
25             match self.0 {
26                 0x0001 => "TPM".to_string(),
27                 0x0002 => "SGX".to_string(),
28                 0x0003 => "SEV".to_string(),
29                 _ if self.0 >= Self::CUSTOM_START =>
30                     format!("Custom-{:04X}", self.0 - Self::CUSTOM_START),
31                 _ => "Unknown".to_string(),
32             }
33         }
34     }

```

The implementation creates a common type system for various attestation technologies with a newtype struct, `AttestationType`, in the attestation module `src/attestation/mod.rs`. This provides an extensible design that allows developers to create attestation types without interfering with the const values assigned to standardized attestation technologies such as AMD SEV-SNP, Intel SGX, and TPM 2.0. In addition to the const values, it supplies an extensible framework for developers to define their own attestation types. The design of `AttestationType` implementation uses a unique range to identify any `AttestationType` values, enabling developers' own

attestation types to be defined without interfering with the standardized attestation types. To make debugging and logging tasks easier, additional utility methods offer short names and descriptions that are readable by humans.

4.2.2 Attestation Request and Response Structures

```

1      pub struct AttestationRequest {
2          pub nonce: Vec<u8>,
3          pub attestation_type: AttestationType,
4          pub data: Vec<u8>,
5      }
6
7      pub struct AttestationResponse {
8          pub report: Vec<u8>,
9          pub signature: Vec<u8>,
10         pub certificate_chain: Vec<u8>,
11         pub attestation_type: AttestationType,
12     }

```

Within TLS extensions, the fundamental data structures for attestation operations are made to be efficiently serializable and memory-safe. The challenge information sent from one party to another is contained in the `AttestationRequest` structure, which also includes the requested `AttestationType`, technology-specific information that might be needed for report generation (`data`), and a `nonce` for freshness.

The `AttestationResponse`, which includes the attestation `report`, cryptographic `signature`, `certificate_chain` for signature verification, and the `AttestationType` that produced the evidence, is contained in the corresponding `AttestationResponse` structure. These structures provide flexibility for various attestation technologies that may have different data formats and sizes while guaranteeing memory safety by using owned byte vectors for binary data.

```

1      impl Codec<'_> for AttestationRequest {
2          fn encode(&self, bytes: &mut Vec<u8>) {
3              (self.nonce.len() as u16).encode(bytes);
4              bytes.extend_from_slice(&self.nonce);
5
6              (self.attestation_type.0).encode(bytes);
7
8              (self.data.len() as u16).encode(bytes);
9              bytes.extend_from_slice(&self.data);
10         }
11
12         fn read(r: &mut Reader<'_>) -> Result<Self, InvalidMessage> {
13             let nonce_len = u16::read(r)? as usize;
14             let nonce = r.take(nonce_len)
15                 .ok_or(InvalidMessage::MissingData("nonce"))?
16                 .to_vec();
17
18             let attestation_type_raw = u32::read(r)?;
19             let attestation_type = AttestationType(attestation_type_raw);

```

```

20
21         let tech_data_len = u16::read(r)? as usize;
22         let data = r.take(tech_data_len)
23         .ok_or(InvalidMessage::MissingData("data"))?
24         .to_vec();
25
26         Ok(Self {
27             nonce,
28             attestation_type,
29             data,
30         })
31     }
32 }

```

For safe serialization within TLS extensions, both structures implement the `rustls Codec<'a>` trait for encoding and decoding. Length-prefixed fields are used in the encoding format to guarantee correct parsing boundaries and avoid buffer overruns, which have historically troubled C implementations.

4.2.3 Trait-Based Provider Interface

```

1     pub trait AttestationReportGenerator: Debug + Send + Sync {
2         fn get_attestation_type(&self) -> AttestationType;
3         fn generate_report(&self, nonce: &[u8], tech_specific_data: &[u8])
4         -> Result<AttestationResponse, Error>;
5     }
6
7     pub trait AttestationReportVerifier: Debug + Send + Sync {
8         fn get_attestation_type(&self) -> AttestationType;
9         fn verify_report(&self, response: &AttestationResponse,
10             expected_nonce: &[u8])
11         -> Result<bool, Error>;
12     }

```

The interface for attestation functionality is defined by two main characteristics: `AttestationReportVerifier` for systems that can validate attestation evidence from distant peers, and `AttestationReportGenerator` for systems that can generate attestation evidence.

The `get_attestation_type()` method is an important identification mechanism that allows the system to identify which type of attestation technology a provider supports by returning the `AttestationType` that the implementation supports. This provides the handshake logic with a basis to perform compatibility checks prior to performing attestation.

The `generate_report()` method in the `AttestationReportGenerator` trait takes two parameters - `nonce` and `tech_specific_data` that represents technology-specific data, such as PCR selections for a TPM, or enclave measurements for SGX. The function returns a `Result` where a success will yield an `AttestationResponse`, while a failure will yield an `Error` that indicates what went wrong while generating the Report.

The `verify_report()` method in the `AttestationReportVerifier` trait takes the complete `AttestationResponse` to verify, and an `expected_nonce` parameter that represents the initial challenge that should be reflected in the attestation report. The method returns `Result` where the `bool` indicates whether the evidence of attestation is indeed verified and trustworthy, and `Error` provides for a situation in which verification cannot sensibly be completed, such as when data has malfunctioned, there is some failure in the cryptographic algorithm being used, or a related situation.

Compared to callback-based methods used in C implementations, these characteristics offer a number of significant advantages. By limiting the use of void pointer callbacks, which are frequently found in C code, to implementations that are properly typed, the trait system helps to avoid type confusion errors. In order to prevent buffer overflow and use-after-free vulnerabilities, all operations employ safe Rust constructs with bounded slices rather than raw pointers.

4.2.4 Attestation Configuration

```

1  pub struct AttestationConfig {
2      pub request: AttestationRequest,
3      pub verifier: Arc<dyn AttestationReportVerifier>,
4      pub report_generator: Arc<dyn AttestationReportGenerator>,
5  }
```

The `AttestationConfig` provides a centralized coordination point to bring together all attestation components. The `AttestationConfig` serves many important roles in the attestation system. In the first instance, it allows endpoints to function in a bifunctional capacity at the same time. Those endpoints can act as a verifier using the `verifier` field to verify attestation evidence from remote peers as well as an attestee with the `report_generator` field to generate attestation evidence to be sent to remote peers.

```

1  // example attestation configuration
2  let attestation_config = AttestationConfig {
3      request: AttestationRequest::new(
4          generate_nonce(),
5          AttestationType::TPM,
6          pcr_selection.to_vec()
7      ),
8      verifier: Arc<dyn AttestationReportVerifier>,
9      report_generator: Arc<dyn AttestationReportGenerator>,
10 }
```

The `request` field contains a pre-configured `AttestationRequest` that serves as a template for requesting attestation from remote peers. The request template sets the attestation type, a nonce for freshness, and any technology-specific data that is required for report generation.

```

1  pub struct ClientConfig {
2      // existing code
3  }
```

```

4      /// TPM attestation configuration
5      pub attestation_config: Option<AttestationConfig>,
6  }
7
8      impl ClientConfig {
9          /// existing code
10
11         pub fn with_attestation_config(mut self, config:
12             AttestationConfig) -> Self {
13             self.attestation_config = Some(config);
14             self
15         }
16     }

```

The `AttestationConfig` is integrated into both client and server configurations as optional fields allowing the existing Rustls applications to remain backwards compatible. The `with_attestation_config` function allows user to optionally add attestation capabilities to an existing `ClientConfig` or `ServerConfig`.

4.2.5 Channel Binding

```

1      pub fn compute_linking_hash(
2          transcript_hash: &hash::Output,
3          shared_secret_bytes: &[u8],
4          attestation_nonce: &[u8],
5          hash_provider: &'static dyn hash::Hash,
6      ) -> hash::Output {
7          let mut ctx = hash_provider.start();
8
9          /// Include the TLS transcript hash
10         ctx.update(transcript_hash.as_ref());
11
12         /// Include the DHE shared secret
13         ctx.update(shared_secret_bytes);
14
15         /// Include the attestation nonce
16         ctx.update(attestation_nonce);
17
18         ctx.finish()
19     }

```

As explained in section 3.2, to avoid relay attacks and to guarantee cryptographic binding of the TLS session to the attestation reports, the implementation utilizes a channel binding mechanism called a linking hash. The linking hash is constructed from three inputs: the transcript hash of the current TLS session, the shared secret derived from the (DHE), and the attestation nonce. The `compute_linking_hash` function will take those three pieces of information and call the hash provider for the provided TLS cipher suite to apply hashing to the components sequentially.

```

1      pub(crate) struct KeyScheduleHandshakeStart {
2          ks: KeySchedule,

```

```

3         dhe_secret_bytes: Option<Vec<u8>>,
4     }
5
6     impl KeyScheduleHandshakeStart {
7         pub(crate) fn into_handshake(
8             mut self,
9             shared_secret: SharedSecret,
10        ) -> KeyScheduleHandshakeStart {
11            let secret_bytes = shared_secret.secret_bytes();
12
13            // Store DHE secret for later linking hash computation
14            let dhe_secret_bytes = Some(secret_bytes.to_vec());
15
16            self.ks.input_secret(secret_bytes);
17
18            KeyScheduleHandshakeStart {
19                ks: self.ks,
20                dhe_secret_bytes,
21            }
22        }
23    }
24
25    pub(crate) struct KeyScheduleHandshake {
26        ks: KeySchedule,
27        client_handshake_traffic_secret: OkmBlock,
28        server_handshake_traffic_secret: OkmBlock,
29        dhe_secret_bytes: Option<Vec<u8>>,
30    }
31
32    impl KeyScheduleHandshake {
33        pub(crate) fn get_dhe_secret(&self) -> Option<&[u8]> {
34            self.dhe_secret_bytes.as_deref()
35        }
36    }
37
38    pub(crate) struct KeyScheduleTrafficWithClientFinishedPending {
39        handshake_client_traffic_secret: OkmBlock,
40        traffic: KeyScheduleTraffic,
41        dhe_secret_bytes: Option<Vec<u8>>,
42    }
43
44    impl KeyScheduleTrafficWithClientFinishedPending {
45        pub(crate) fn get_dhe_secret(&self) -> Option<&[u8]> {
46            self.dhe_secret_bytes.as_deref()
47        }
48
49        pub(crate) fn clear_dhe_secret(&mut self) {
50            if let Some(ref mut secret) = self.dhe_secret_bytes {
51                secret.zeroize();
52            }
53            self.dhe_secret_bytes = None;
54        }
55    }

```

In order to enable linking hash computation, the key schedule retains the DHE secret beyond its typical lifetime. The DHE shared secret is generally consumed during the key derivation, but for attestation, it must remain around until the certificate processing (which includes the attestation operations) is done. The DHE secret is passed along the key schedule hierarchy so that it is retrievable for each step of TLS handshake and attestation process (`KeyScheduleHandshakeStart` $\rightarrow$ `KeyScheduleHandshake` $\rightarrow$ `KeyScheduleTrafficWithClientFinishedPending`). After the last attestation verification is complete (server verified client's report), the implementation provided a method to securely clean up DHE secret to assure that it doesn't exist in memory. The `clear_dhe_secret()` use the zeroize crate to zero out the secret bytes in memory prior to deallocating the vector when no longer needed. This guards against potential memory disclosure attacks and prevents the DHE secret from being retained in memory longer than needed.

```

1      // In server/hs.rs - ExtensionProcessing::process_common
2      if let Some(client_request) = hello.get_attestation_request() {
3          if let Some(dhe_secret_bytes) = ks.get_dhe_secret() {
4              let transcript_hash = transcript.current_hash();
5
6              // Compute linking hash using client's nonce
7              let linking_hash = crate::attestation::compute_linking_hash(
8                  &transcript_hash,
9                  dhe_secret_bytes,
10                 &client_request.nonce, // Client's original nonce
11                 config.provider.cipher_suites[0].hash_provider(),
12             );
13
14             // Generate server's attestation report bound to this TLS
15             // session
16             let report =
17                 attestation_config.report_generator.generate_report(
18                     &client_request.data, // Client-specified technology data
19                     linking_hash.as_ref() // Session-bound linking hash
20                 )?;
21             self.exts.push(ServerExtension::AttestationResponse(report));
22         }
23     }
24
25     // In server/tls13.rs - ExpectCertificate handler
26     if let Some(dhe_secret_bytes) = self.key_schedule.get_dhe_secret() {
27         // Use preserved transcript hash from certificate request time
28         let transcript_hash = cx.data.transcript_hash_after_cert_request
29             .as_ref()
30             .ok_or_else(|| {
31                 Error::AttestationVerificationFailed(
32                     "No stored transcript hash available for client attestation
33                     verification".to_string()
34                 )
35             })?;
36
37         // Compute linking hash using server's nonce (from server's
38         // request)

```

```

36     let linking_hash = crate::attestation::compute_linking_hash(
37         &transcript_hash,
38         dhe_secret_bytes,
39         &attestation_config.request.nonce, // Server's nonce for client
           attestation
40     self.suite.common.hash_provider,
41     );
42
43     // Clear sensitive material after use
44     self.key_schedule.clear_dhe_secret();
45     cx.data.transcript_hash_after_cert_request = None;
46
47     // Verify client's attestation report
48     let verification_result =
49         attestation_config.verifier.verify_report(
50         client_response,
51         linking_hash.as_ref()
52     )?;
53
54     if !verification_result {
55         return Err(cx.common.send_fatal_alert(
56             AlertDescription::BadCertificate,
57             Error::AttestationVerificationFailed("Client quote
           verification failed".to_string()),
58         ));
59     }
60
61     debug!("Client quote verification SUCCESSFUL");

```

The linking hash is then computed everytime an attestation operation (`generate_report()` or `verify_report()`) needs to be done. In the code snippet above the linking hash is computed for the server report generation in `ExtensionProcessing::process_common` where the server report generation takes place and in `ExpectCertificate` where the client attestation report is verified by the server. Similar approach is done for the server report verification by client (`ExpectEncryptedExtensions`) and client report generation (`ClientAuthDetails::resolve`). `clear_dhe_secret` is called in `ExpectCertificate` because this is the last point where the dhe secret is needed for the linking hash computation.

4.3 Client-Side Integration

```

1     // In client/client_conn.rs
2     pub struct ClientConfig {
3         // existing configuration fields...
4
5         /// Remote attestation configuration
6         pub attestation_config: Option<AttestationConfig>,
7     }
8
9     impl ClientConfig {

```

```

10         // existing methods...
11
12         pub fn with_attestation_config(mut self, config:
13             AttestationConfig) -> Self {
14             self.attestation_config = Some(config);
15             self
16         }

```

The client-side implementation introduces attestation support by modifying configuration and handshake processing but maintains backward compatibility for existing applications using Rustls. This means an application can add attestation support as a new feature while maintaining existing functionality through the `ClientConfig` abstraction. Applications can add attestation support by extending `ClientConfig` with an attestation configuration that is optional. An application simply needs to call `with_attestation_config()` with corresponding attestation configuration, which then includes the attestation request template, report generator, and verifier to enable attestation.

```

1         // In client/hs.rs - emit_client_hello_for_retry
2         // add attestation request to client hello
3         if let Some(attestation_config) = &config.attestation_config {
4             exts.push(ClientExtension::AttestationRequest(
5                 attestation_config.request.clone()));
6         }

```

When attestation is configured, the client implementation adds attestation request as an extension to the `ClientHello` message during handshake processing, which signals the server that it wishes to receive an attestation response.

```

1         // In client/tls13.rs - ExpectEncryptedExtensions
2         let client_sent_attestation_request = self.hello.sent_extensions
3             .contains(&ExtensionType::AttestationRequest);
4
5         for ext in exts.iter() {
6             match ext {
7                 ServerExtension::AttestationResponse(attestation_response) =>
8                 {
9                     if client_sent_attestation_request {
10                         debug!("Received attestation response from server");
11
12                         if let Some(ref attestation_config) =
13                             self.config.attestation_config {
14                             // Verify attestation type compatibility
15                             if attestation_config.verifier.
16                                 get_attestation_type() ==
17                                 attestation_response.attestation_type {
18                                 if let Some(dhe_secret_bytes) =
19                                     self.key_schedule.get_dhe_secret() {
20
21                                     // Compute linking hash for verification
22                                     let linking_hash = crate::attestation::

```



```

19         compute_linking_hash(
20             &transcript_hash,
21             dhe_secret_bytes,
22             &attestation_config.request.nonce,
23             self.suite.common.hash_provider,
24         );
25
26         // Verify server's attestation quote
27         let is_valid = attestation_config.verifier
28             .verify_report(
29                 attestation_response,
30                 linking_hash.as_ref()
31             )?;
32
33         if !is_valid {
34             return Err(cx.common.send_fatal_alert(
35                 AlertDescription::BadCertificate,
36                 Error::Attestation
37                 VerificationFailed("Server
38                     attestation verification
39                     failed".to_string()),
40             ));
41         }
42
43         debug!("Server attestation verification
44             successful");
45     }
46 } else {
47     return Err(cx.common.send_fatal_alert(
48         AlertDescription::BadCertificate,
49         Error::AttestationVerificationFailed("Client
50             attestation type mismatch".to_string()),
51     ));
52 }
53 }
54 } else {
55     // Reject unsolicited attestation responses
56     return Err(cx.common.send_fatal_alert(
57         AlertDescription::UnsupportedExtension,
58         PeerMisbehaved::UnsolicitedEncryptedExtension,
59     ));
60 }
61 }
62 }
63 }

```

During the `EncryptedExtensions` handling phase, the client processes server-attestation-related extension messages. Each processing phase of responses includes some important security checks. First, the client ensures that the client will only process `AttestationResponse` from the server if the client sent a matching request. Second, the client will verify that the attestation type from server matches the attestation type that the clients verifier is capable of executing. Third, the client computes the linking hash to check that the attestation report by the server is cryptographically bound to

the current TLS session. Finally, the configured verifier checks the attestation evidence according to the chosen attestation platform.

```

1      // In client/client_conn.rs
2      pub struct ClientConnectionData {
3          pub(super) early_data: EarlyData,
4          pub(super) ech_status: EchStatus,
5          pub(super) server_attestation_request: Option<AttestationRequest>,
6      }
7
8      // In client/tls13.rs - ExpectEncryptedExtensions
9      // process attestation request from server
10     ServerExtension::AttestationRequest(attestation_request) => {
11         if client_sent_attestation_request {
12             debug!("Received attestation request from server for mutual
13                 attestation");
14             debug!("Server requested attestation type: {:?}",
15                 attestation_request.attestation_type);
16
17             server_attestation_request =
18                 Some(attestation_request.clone());
19         } else {
20             return Err(cx.common.send_fatal_alert(
21                 AlertDescription::UnsupportedExtension,
22                 PeerMisbehaved::UnsolicitedEncryptedExtension,
23             ));
24         }
25     }
26
27     //assign server attestation request to
28     ClientConnectionData.server_attestation_request
29     if let Some(request) = server_attestation_request {
30         cx.data.server_attestation_request = Some(request);
31     }

```

In mutual attestation scenarios, the client also processes attestation requests from the server. The server attestation request is saved in the `ClientConnectionData` and referenced later when the client processes the certificate, at which point the client will perform its attestation report.

```

1      // In client/common.rs - ClientAuthDetails::resolve
2      // generate client's attestation response
3      let attestation_response = if let (Some(config),
4          Some(server_request)) =
5          (attestation_config, server_attestation_request) {
6          if config.report_generator.get_attestation_type() ==
7              server_request.attestation_type {
8
9              if let (Some(transcript), Some(secret), Some(hash_prov)) =
10                  (transcript_hash, dhe_secret, hash_provider) {
11
12                  // Compute linking hash using server's nonce
13                  let linking_hash =
14                      crate::attestation::compute_linking_hash(

```

```

12         transcript,
13         secret,
14         &server_request.nonce, // Server's nonce for client
           attestation
15         hash_prov,
16     );
17
18     // Generate client's attestation report bound to this TLS
           session
19     Some(config.report_generator.generate_report(
20         &server_request.data,
21         linking_hash.as_ref()
22     )?)
23 } else {
24     error!("Missing parameters for linking hash computation");
25     return Err(Error::AttestationGenerationFailed(
26         "Cannot compute linking hash: missing transcript, DHE
           secret, or hash provider".to_string()
27     ));
28 }
29 } else {
30     debug!("Server requested different attestation type than
           client supports");
31     None
32 }
33 } else {
34     None
35 };

```

The `ClientAuthDetails::resolve` method takes on the work of generating the client's attestation response by verifying that all required parts are present and compatible via a series of conditional checks. First, it verifies that both an attestation configuration and a server attestation request are present, confirming mutual attestation is required. Similar to the server's `AttestationReport` verification in `ExpectEncryptedExtensions`, it makes sure that the attestation types between the client's report generator and the server's request are compatible. Finally, it confirms that all of the required cryptographic components for linking hash computation, which are the transcript hash, DHE secret, and hash provider, are present.

```

1 // In client/tls13.rs - emit_certificate_tls13
2 for (i, cert) in certs.iter().enumerate() {
3     let mut extensions = Vec::new();
4
5     // Add attestation response to the first certificate entry
6     if i == 0 {
7         if let Some(response) = attestation_response {
8             extensions.push(CertificateExtension::
9                 AttestationResponse(response.clone()));
10            debug!("Client's attestation response added to
              certificate entry as extension");
11        }
12    }
13 }

```

```

14         cert_entries.push(CertificateEntry {
15             cert: cert.clone(),
16             exts: extensions,
17         });
18     }

```

The client's `AttestationResponse` is then added as an extension in the client certificate message.

4.4 Server-Side Integration

The `ServerConfig` follows similar configuration pattern as the `ClientConfig`. The server report generation takes place in `ExtensionProcessing::process_common` and the verification of the client report takes place in `ExpectCertificate`. Both works the same as the client report generation and server report verification in the client side.

```

1  // In client/tls13.rs - validate_encrypted_extensions
2  fn validate_encrypted_extensions(
3      common: &mut CommonState,
4      hello: &ClientHelloDetails,
5      exts: &Vec<ServerExtension>,
6  ) -> Result<(), Error> {
7      // existing code
8
9      for ext in exts.iter() {
10         match ext {
11             // If client doesn't send attestation request, then
12             // server should not send back attestation response
13             ServerExtension::AttestationResponse(_) => {
14                 if !hello.sent_extensions.contains(
15                     &ExtensionType::AttestationRequest) {
16                     return Err(common.send_fatal_alert(
17                         AlertDescription::UnsupportedExtension,
18                         PeerMisbehaved::UnsolicitedEncryptedExtension,
19                     ));
20                 }
21             }
22             // If client doesn't send attestation request, then
23             // server should not send back attestation request
24             ServerExtension::AttestationRequest(_) => {
25                 if !hello.sent_extensions.contains(
26                     &ExtensionType::AttestationRequest) {
27                     return Err(common.send_fatal_alert(
28                         AlertDescription::UnsupportedExtension,
29                         PeerMisbehaved::UnsolicitedEncryptedExtension,
30                     ));
31                 }
32             }
33         }
34     }

```

```

35         Ok(())
36     }

```

The server sends `AttestationReport` and `AttestationRequest` in the form of encrypted extensions. The `validate_encrypted_extensions` is updated to ensure that the server only sends these attestation details in the case that the client actually requires mutual attestation.

```

1  // In server/server_conn.rs
2  pub struct ServerConnectionData {
3      pub(super) sni: Option<DnsName<'static>>,
4      pub(super) received_resumption_data: Option<Vec<u8>>,
5      pub(super) resumption_data: Vec<u8>,
6      pub(super) early_data: EarlyDataState,
7      pub(super) client_attestation_request: Option<AttestationRequest>,
8      pub(super) transcript_hash_after_cert_request:
9          Option<hash::Output>,
10 }
11
12 // In server/tls13.rs - emit_certificate_req_tls13
13 // Includes the CertificateRequest in transcript hash
14 let creq = HandshakeMessagePayload(HandshakePayload::
15     CertificateRequestTls13(cr));
16
17 trace!("Sending CertificateRequest {creq:?}");
18 flight.add(creq);
19
20 cx.data.transcript_hash_after_cert_request =
21     Some(flight.transcript.current_hash());
22
23 // In client/tls13.rs - ExpectCertificateRequest
24 // Includes CertificateRequest in transcript hash
25 let certreq = &require_handshake_msg!(
26     m,
27     HandshakeType::CertificateRequest,
28     HandshakePayload::CertificateRequestTls13
29 )?;
30 self.transcript.add_message(&m);
31 debug!("Got CertificateRequest {certreq:?}");
32
33 let transcript_hash = self.transcript.current_hash();
34
35 // In server/tls13.rs - ExpectCertificate
36 // Will include Certificate message and lead to verification error
37 // when used
38 let transcript_hash = transcript.current_hash();
39
40 // Solution: in server/tls13.rs - emit_certificate_req_tls13
41 let creq = HandshakeMessagePayload(HandshakePayload::
42     CertificateRequestTls13(cr));
43
44 trace!("Sending CertificateRequest {creq:?}");
45 flight.add(creq);
46

```

```

44 // Transcript hash containing messages up to CertificateRequest,
    similar to the one used in ExpectCertificateRequest
45 cx.data.transcript_hash_after_cert_request =
    Some(flight.transcript.current_hash());

```

One of the main difference between the client-side integration and server-side integration is the preservation of transcript hash state for later client attestation verification in the server-side. The issue arises during the transcript hash computation process. Once a client receives the `CertificateRequest` message, it adds this message to the transcript and computes the linking hash for its attestation report. However, if `transcript_hash.current_hash` were to be used in `ExpectCertificate`, the server will verify the attestation report with the transcript hash that contains the client's certificate message due to the `flight.add()` call in `emit_certificate_tls13`, while the client's linking hash only included the `CertificateRequest` message. Both links need to have the same transcript content as input when creating the linking hash, resulting in verification failure. The addition of the preserved `transcript_hash_after_cert_request` prevents this issue since we are caching transcript state immediately after the client sends its `CertificateRequest` to the server and this matches exactly with what the client provided to the server. When the server goes to verify the client's attestation response, it is now using the preserved hash instead of the current transcript hash.

4.5 TPM Implementation

The implementation includes a dedicated TPM 2.0 attestation to demonstrates how the abstract attestation interfaces can be implemented for an attestation provider. The TPM implementation consists of two main components: `TpmReportGenerator` for generating TPM quotes and `TpmReportVerifier` for verifying the quotes.

4.5.1 TPM Report Generation

```

1 // In attestation/tpm.rs
2 impl AttestationReportGenerator for TpmReportGenerator {
3     fn get_attestation_type(&self) -> AttestationType {
4         AttestationType::TPM
5     }
6
7     fn generate_report(&self, pcr_selection: &[u8], linking_hash:
        &[u8]) -> Result<AttestationResponse, Error> {
8         let (report, signature, ak_cert) =
9             get_tpm_report(linking_hash, pcr_selection)?;
10
11         Ok(AttestationResponse::new(
12             report,
13             signature,
14             ak_cert,
15             self.get_attestation_type(),
16         ))

```

```

16     }
17 }
18
19 pub fn get_tpm_report(linking_hash: &[u8], pcr_selection: &[u8]) ->
20     Result<(Vec<u8>, Vec<u8>, Vec<u8>), Error> {
21     read_with_attestation_key(linking_hash, pcr_selection)
22     .map_err(|e| {
23         error!("TPM attestation failed: {:?}", e);
24         Error::AttestationGenerationFailed(format!("TPM attestation
25             failed: {:?}", e))
26     })
27 }
28
29 fn read_with_attestation_key(linking_hash: &[u8], pcr_selection:
30     &[u8]) -> Result<(Vec<u8>, Vec<u8>, Vec<u8>), Error> {
31     // configures the connection to use a software TPM simulator
32     let tcti = TctiNameConf::Swtpm(Default::default());
33     let mut ctx = Context::new(tcti)?;
34
35     //create EK and derive AK
36     let ak_handle = create_attestation_key(&mut ctx)?;
37
38     let pcr_slots: Vec<PcrSlot> = pcr_selection
39         .iter()
40         .filter_map(|&i| PcrSlot::try_from(i as u32).ok())
41         .collect();
42
43     let pcr_selection_list = PcrSelectionListBuilder::new()
44         .with_selection(HashingAlgorithm::Sha256, &pcr_slots)
45         .build()?;
46
47     let qualifying_data = Data::try_from(linking_hash.to_vec())?;
48     let signature_scheme = SignatureScheme::RsaPss {
49         hash_scheme: HashScheme::new(HashingAlgorithm::Sha256),
50     };
51
52     let (quoted_data, signature) = ctx
53         .execute_with_nullauth_session(|ctx| {
54             ctx.quote(ak_handle, qualifying_data, signature_scheme,
55                 pcr_selection_list)
56         })?;
57
58     let (ak_public, _, _) = ctx
59         .execute_with_nullauth_session(|ctx| ctx.read_public(ak_handle))?;
60
61     let quoted_bytes = quoted_data.marshall()?;
62     let signature_bytes = signature.marshall()?;
63     let ak_public_bytes = ak_public.marshall()?;
64
65     // Include pcr selection and linking hash in TPM quote
66     let mut report = Vec::new();
67     report.extend_from_slice(linking_hash);
68     report.extend_from_slice(&quoted_bytes);
69
70     Ok((report, signature_bytes, ak_public_bytes))

```

```
67     }
```

The quote generation mechanism involves establishing an attestation key hierarchy, selecting the Platform Configuration Registers (PCR), and producing a cryptographically signed quote which includes the linking hash as qualifying data. In this implementation, the `tss-esapi` crate is used to enable interaction to the TPM instance (in this case the Software TPM instance) via the standard TSS 2.0 Enhanced System API. The function `get_tpm_report` generates an endorsement key, derives an attestation key from it, and then uses the attestation key to generate a TPM quote over the selected PCRs with the linking hash as qualifying data.

4.5.2 TPM Report Verification

```
1  impl AttestationReportVerifier for TpmReportVerifier {
2      fn get_attestation_type(&self) -> AttestationType {
3          AttestationType::TPM
4      }
5
6      fn verify_report(&self, response: &AttestationResponse,
7                      expected_linking_hash: &[u8]) -> Result<bool, Error> {
8          verify_tpm_report(
9              &response.report,
10             &response.signature,
11             &response.certificate_chain,
12             expected_linking_hash
13         )
14     }
15
16     pub fn verify_tpm_report(
17         report: &[u8],
18         signature: &[u8],
19         ak_public_bytes: &[u8],
20         expected_linking_hash: &[u8]
21     ) -> Result<bool, Error> {
22         // handle edge cases
23         if signature.is_empty() || ak_public_bytes.is_empty() ||
24            report.is_empty() {
25             warn!("Invalid input: empty signature, public key, or
26                report");
27             return Ok(false);
28         }
29
30         // Check linking hash validity
31         if !verify_linking_hash(report, expected_linking_hash)? {
32             warn!("Quote verification failed: linking hash mismatch");
33             return Ok(false);
34         }
35
36         // Check TPM
37         let (attest, signature_struct, ak_public) = match
38             parse_tpm_structures(
```



```

36     report,
37     signature,
38     ak_public_bytes,
39     expected_linking_hash
40 ) {
41     Ok(structures) => structures,
42     Err(e) => {
43         warn!("Failed to parse TPM structures: {:?}", e);
44         return Ok(false);
45     }
46 };
47
48 // Check signature validity
49 if !verify_quote_signature(&attest, &signature_struct,
50     &ak_public)? {
51     warn!("Quote verification failed: invalid signature");
52     return Ok(false);
53 }
54
55 // Check pcr information validity
56 if !validate_pcr_digest(&attest)? {
57     warn!("Quote verification failed: PCR validation failed");
58     return Ok(false);
59 }
60
61 info!("TPM quote verification successful");
62 Ok(true)
63 }

```

The verifier checks TPM quotes using a multi-step process that checks linking hash correspondence, parses TPM attestation structures, verifies the RSA-PSS signature with the attestation key, and checks the PCR digest consistency. The `verify_tpm_report` function first extracts and verifies the linking hash in the report, then unmarshalls the TPM attestation structures, acquires the RSA public key from the attestation key, and does cryptographic signature verification with RSA-PSS with SHA-256. This way, the verification could ensure that the quote was generated for the TLS session.

5 Evaluation

5.1 Performance

5.1.1 Benchmarking

To assess the performance overhead related to adding TPM-based remote attestation to the TLS handshake, a benchmarking analysis of three scenarios- unaltered TLS handshakes, TLS handshakes with server-only TPM attestation, and TLS handshakes with mutual TPM attestation overall was performed. All tests utilized TLS 1.3 with certificate authentication and measured completion time for end-to-end handshakes over 100 iterations of client-server connections to maintain statistical validity and account for system variability.

All benchmarks are conducted on a Huawei MateBook 14s. The specific hardware consists of an Intel Core i5-11300H processor at a base frequency of 3.10 GHz, 8 GB of LPDDR4 memory running at 3733 MHz in dual-channel configuration over 8 memory banks, and 512GB NVme SSD. The system is running Ubuntu 22.04 LTS, with kernel version 6.8.0-65-generic, on the ext4 file system. The benchmark was run using software TPM (swtpm) instances to simulate the function of an actual TPM 2.0 to represent client and server environments.

The performance evaluation measured timing metrics across all test scenarios to understand the performance implications of each attestation configuration. Total handshake completion time captures the end-to-end latency from connection initiation to handshake completion, providing the most critical metric for understanding user-perceived performance impact. TPM report generation time measures the duration required for TPM quote generation, including Platform Configuration Register reading, cryptographic hashing operations, and digital signing using TPM-protected attestation keys. TPM report verification time quantifies the processing required for signature verification, nonce validation, and structural integrity checks performed by the receiving party. TLS + attestation messages accounts for the processing time required for the remote attested TLS messages (excluding the quote generation and verification).

5.1.2 Latencies

The handshake latencies, both on the client and server side, of the 3 scenarios are shown in Table 1.

To better understand performance overhead concerning the additional time involved with TPM, measurements of individual components of the extended handshake approach

Table 1: TLS Handshake Performance Comparison

Scenario	Side	Mean (ms)	Std Dev (ms)
Pure TLS	Server	2.52	0.4
	Client	1.70	0.3
TLS + Server Attestation	Server	244.20	84.9
	Client	243.91	84.8
TLS + Mutual Attestation	Server	520.58	175.4
	Client	516.53	175.1

are performed. Table 2 depicts the breakdown of the latency for TLS + server TPM attestation and table 3 depicts the breakdown of the latency for TLS + mutual TPM attestation.

Table 2: TLS + Server TPM Attestation Handshake Latency Breakdown

Component	Server (ms)	Std Dev (ms)	Client (ms)	Std Dev (ms)
Total Handshake	244.20	84.9	243.91	84.8
TPM Report Generation	238.66	84.8	0.00	0.00
TPM Report Verification	0.00	0.00	3.27	0.5
TLS + Attestation Messages	2.25	0.40	1.81	0.34

Table 3: TLS + Mutual TPM Attestation Handshake Latency Breakdown

Component	Server (ms)	Std Dev (ms)	Client (ms)	Std Dev (ms)
Total Handshake	520.58	175.4	516.53	175.1
TPM Report Generation	254.15	108.3	256.96	120.6
TPM Report Verification	3.42	0.5	3.42	0.6
TLS + Attestation Messages	2.39	0.23	2.00	0.4

5.1.3 Analysis

5.1.3.1 Overhead Magnitude

As shown on table 3, on the server side this caused an increase in handshake time of $207\times$ (from 2.52ms to 520.58ms). The client-side is even more significant with a $304\times$ increase (from 1.7ms to 516.53ms). This means that for both sides, they are looking at an additional latency of more less half a second which is a considerable performance change of the TLS handshake.

The server-only attestation results show an agreement with the mutual attestation data. The server-only scenario achieves about half the latency of mutual attestation (244.20 milliseconds compared to 520.58 milliseconds), and this was the expected theoretical

performance because the server-only attestation uses all of the protocol without the overhead of generating a TPM report on the client side and verifying the report on server side. This near perfect halving of latency supports the idea that both sets of measurements are both reliable and accurate by demonstrating that the performance overhead scales accordingly to the number of attesting parties.

The variability of software TPM implementations is reflected in the standard deviations of the TPM-attested configurations. In contrast to hardware TPMs, which have cryptographic processors and relatively fixed execution environments, software TPM must share resources with other system processes. Variations in performance occur with different CPU loads, memory usage, and system scheduling. The standard deviation for TPM report generation is most significant due to the complex operations involving PCR reading, hashing, and digital signing operations, which contends for resources at the system-level.

TPM Report Generation The major performance bottleneck is TPM quote generation, which is responsible for nearly 50% of handshake time. This includes reading and hashing Platform Configuration Registers (PCRs), performing cryptographic signing operations using TPM-protected attestation keys, and generating comprehensive platform state measurement reports. These operations require heavy computational power which leads to the massive overhead.

TPM Report Verification Unlike the reports generation, the overhead for quote verification is very small at about 3-4ms per quote. Verification involves signature verification with RSA-PSS and SHA-256, nonce verification and the validation of the PCR selections. Due to the nature of TPM report verification being cryptographically lightweight the overhead for this phase of attestation is barely noticeable in comparison to the report generation which not only requires multiple measurements from various components of the system but also multiple cryptographic operations to generate the quotes and Attestation Keys (AK).

TLS Protocol Extensions The TLS protocol extension shows no impact on the overall handshake performance. The extended handshake messages seems to have the same amount of processing time as the pure TLS messages (more less 2 seconds). The processing times between TLS extension and pure TLS baseline performance are so similar which shows that by adding attestation, the performance of the TLS 1.3 protocol is fully retained. Most of the overhead can be attributed to TPM operations and not protocol responsible overhead.

5.1.3.2 Comparison with Prior Work

The performance characteristics of our SWTPM implementation differ significantly from hardware TPM implementations due to architectural differences. SWTPMs must compete

for system resources with other processes and lack dedicated cryptographic processors, leading to higher variability and different performance profiles compared to hardware TPMs that operate with dedicated cryptographic processors and consistent execution environments. This distinction influences both absolute performance and measurement consistency when comparing our results with hardware-based implementations.

In the RATLS [WWR22] evaluation, the Infineon Optiga SLB 9670 TPM 2.0 had an average of about 525ms to perform an initial handshake on the client-side versus a baseline average of 52ms for a pure TLS. Taking an average client-side initial handshake time of 525ms from the RATLS work and compare it to their baseline of 52ms we can see an overhead factor of approximately 10x. The rustls + swtpm implementation achieves more less the same latency as RATLS. The SWTPM implementation also shows substantially higher variance, meaning less predictable performance compared to dedicated hardware.

One thing certain is that all the measurements from this paper, RATLS, and TLS+RA show that remote attested TLS has considerably more overhead than a pure TLS handshake.

5.1.4 Deployment Considerations

Connection Establishment Latency The additional latency in connection establishment has a large effect for interactive applications because this time is sustained on the immediate time response experienced by the user. For many applications, this overhead may be justified if they are leaning towards security and performance is less important, or in situations with long-lived connections and many data exchanges where the connection handshake delay could be tolerated.

Hardware TPM Reliability Deployment scenarios with hardware TPMs would show more reliable and predictable performance characteristics from a practical perspective compared to software TPMs. This is due to a number of key advantages presented by hardware TPM implementations: a dedicated cryptographic processor was used and had no resource contention with system processes, they also offer consistent execution times that reduce the high variability we observed in the outcomes for the software, and the isolated execution environment is free from interference from the fluctuation in the system load. For the deployment of applications with stricter security guarantees, hardware TPMs have even greater security assurances because of stricter layers of assurance, evidenced by reference monitors or the use of a tamper-resistant piece of hardware to prevent software-based attacks toward the cryptographic operations and overall system compromise. Because of these unique characteristics, hardware TPMs would present a viable solution for deployment in production environments needing to accommodate the dual requirement of predictable performance levels and security assurances.

5.2 Security

The implementation satisfies all security requirements as specified in section 3.2 via integration tests in the `security_tests.rs` test module in the `rustls` crate. Two separate software TPM (`swtpm`) instances were used for the tests. In the test configurations, one `swtpm` instance acts as the server on port 2323, and the other as the client on port 2321, which allows realistic simulation of mutual attestation conditions.

5.2.1 Maintain TLS Security Properties

The `test_tls_security_properties_preserved_with_attestation` asserts that when adding attestation-specific functionality to TLS, the standard TLS protections are preserved. The test passes for confidentiality, as receiving sensitive data sent over an encrypted channel will assert that the data received is entirely unchanged with `assert_eq!(received_sensitive_data, sensitive_data)`. The integrity protection is validated by ensuring the data sent can be validated by verifying the bytes received with `assert_eq!(received_data, &integrity_data_clone[..])`. Lastly, authentication is validated by using `assert!(tls_stream.conn.peer_certificates().is_some())` to validate the peer's certificate. This ensures that the original three fundamental security guarantees of TLS: confidentiality, integrity, and authentication remain intact while implementing attestation to the application protocol.

5.2.2 Mutual Attestation Support

The ability for clients and servers to simultaneously attest to their software stacks was made possible by the implementation's complete mutual attestation capabilities. The `test_mutual_attestation_with_swtpm` function illustrates this ability. The test sets up both endpoints using respective `AttestationConfig` structures that include `TpmReportGenerator` for generating the attestation report and `TpmReportVerifier` for validating the received report. Each party makes its own nonces and selects PCRs; the client uses its own `client_nonce` and identical PCR selection as the server, and the server uses `server_nonce` and `pcr_selection = vec![1, 2, 4, 8]` (power of 2 due to `tss-esapi` bitmask PCR selection representation). The test verifies successful bidirectional attestation by verifying that the client and server both verified the peer's attestation quote before establishing the secure communication channel.

5.2.3 Channel Binding

```
1      struct LinkingHashTamperingVerifier;
2
3      impl AttestationReportVerifier for LinkingHashTamperingVerifier {
4          fn verify_report(&self, response: &AttestationResponse,
5                          expected_linking_hash: &[u8]) -> Result<bool, Error> {
6              let mut tampered_hash = expected_linking_hash.to_vec();
7              if tampered_hash.len() > 0 {
6                  tampered_hash[0] ^= 0xFF; // Flip bits in first byte
```

```

8         }
9         verify_tpm_report(&response.report, &response.signature,
10                          &response.certificate_chain, &tampered_hash)
11     }

```

The implementation establishes cryptographic binding between the TLS session and attestation evidence based on a linking hash. The test illustrates this binding by using a custom `LinkingHashTamperingVerifier` that intentionally utilized manipulated linking hash during verification in the `test_linking_hash_verification_failure_terminates_connection` test case. The connection is successfully terminated upon using this corrupted verifier. This demonstrates that linking hash verification is functioning properly.

5.2.4 Secure Failure

```

1     struct FailingTpmReportVerifier;
2
3     impl AttestationReportVerifier for FailingTpmReportVerifier {
4         fn verify_report(&self, _response: &AttestationResponse,
5                          _expected_linking_hash: &[u8]) -> Result<bool, Error> {
6             Ok(false)
7         }
8     }

```

The implementation handles attestation failures correctly as it terminates connections when report verification fails. This is observed in the `test_attestation_failure_terminates_connection` method. The test in this method uses a `FailingTpmReportVerifier` that always returns false when verifying. In this test, when the failing verifier is configured on the server side, any attempt to start a communication session results in termination.

6 Future Work

The remote attestation integration with TLS done in this work shows that it is possible to have attestation and memory-safe cryptographic implementations work together. Based on this work, additional research could take the work in many directions to support more deployment scenarios, improve performance characteristics, and improve security properties.

Hardware Attestation For this work, the software based attestation simulation is used for development and testing, but this adds significant performance overhead. Future work should address this and work toward optimizing the integration with hardware security modules to produce production-ready performance characteristics. This includes investigating the performance implications of different hardware attestation technologies because each hardware platform presents unique optimization opportunities.

Attestation Caching and Session Resumption While the current implementation supports basic session resumption, future implementations could include more sophisticated caching techniques for attestation evidence. This includes developing time-bound attestation caches that support reuse of recent attestation reports from a sensible time frame, developing protocols to reuse and re-attest only changed portions of the system that need to be updated, and developing secure session resumption mechanisms that provide the desired attestation guarantees for re-establishments of the connections [Sal+08].

Heterogenous Attestation Environments Heterogeneous environments are common when deploying in the real world. There may be different endpoints that utilize different attestation technologies. Future work should tackle cross-platform cross-support attestation scenarios that consist of developing negotiation protocols for endpoints to discover and agree on compatible attestation mechanisms [Dus+22].

Dynamic Attestation Policies Current attestation verification is based on the static configuration of the various trusted platform states. Future directions may explore possibly developing dynamic policy framework to implement more flexible forms of attestation verification and integrating machine learning models for anomaly detection to identify suspicious platform states [YH24].

7 Conclusion And Outlook

The work has proven that memory-safe remote attestation can be implemented with TLS without any loss in the security properties or protocol compatibility established in previous work. The implementation provides a solution for the memory safety vulnerabilities found in C-based cryptographic applications while preserving the existing intra-handshake methodology for attestation integration.

The implementation shows that memory safety and remote attestation are not mutually exclusive. Using Rustls as a base and a trait-based architecture, this work accomplishes both strong security guarantees and extensible support for multiple remote attestation technologies. The implementation successfully managed to prevent memory-corruption vulnerabilities while preserving full protocol compatibility with other remote attestation platforms.

The results of the evaluation demonstrate significant trade-offs between security and performance. The attestation integration does create significant overhead relative to standard TLS, in exchange for the additional security measurements.

In contrast to previous implementations, the memory safe method had similar performance characteristic while providing stronger implementation-level security (in terms of memory safety). As the RATLS implementation using OpenSSL and hardware TPM had similar latency patterns, it can be concluded that any performance penalties are simply a function of executing an attestation process, not the implementation language, or choice of cryptographic library.

The security analysis validates that the implementation handled the attack vectors that remote attestation systems seek to mitigate. The cryptographic binding from TLS sessions to attestation report prevents relay attacks and the multi-layered validation ensures the integrity of the attested data throughout the attestation process.

At a high level architecture, the traits based design shows that clean abstractions are possible in complex security protocols. Ensuring a distinction between the generic protocol logic and the platform specific implementations is crucial to the system maintenance. In addition to that, this enables the integration of new attestation technologies without major changes to the system as a whole.

The potential application of this work goes far beyond remote attestation. As distributed systems become more complicated and new threats emerge, the correctness of implementation becomes pivotal. This work shows that modern programming languages can be used in established cryptographic protocols with no compromise to security properties and little to no drawback for the overall performance.

Bibliography

- [21] *CVE-2021-3449: OpenSSL TLS server denial-of-service due to NULL pointer dereference*. National Vulnerability Database (NVD). Accessed: August 11, 2025. 2021. URL: <https://nvd.nist.gov/vuln/detail/CVE-2021-3449>.
- [Clo24] Cloudflare. *What is Transport Layer Security (TLS)?* Accessed: August 11, 2025. Cloudflare, Inc., 2024. URL: <https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/>.
- [Cod14] Codenomicon Ltd. *CVE-2014-0160: The Heartbleed Bug*. Accessed: August 11, 2025. 2014. URL: <https://heartbleed.com/>.
- [Con24] Confidential Computing Consortium. *What is Remote Attestation? Enhancing Data Governance with Confidential Computing*. Accessed: August 11, 2025. Confidential Computing Consortium, Oct. 2024. URL: <https://confidentialcomputing.io/2024/10/02/what-is-remote-attestation-enhancing-data-governance-with-confidential-computing/>.
- [Cur20] Cure53. *Security Review & Audit Report: rustls (TLS library written in Rust)*. Tech. rep. Independent security audit of rustls; no critical vulnerabilities found. Cure53, Berlin, June 2020. URL: <https://cure53.de/pentest-report-rustls.pdf>.
- [Cyb24] Cybersecurity and Infrastructure Security Agency. *The Urgent Need for Memory Safety in Software Products*. Government guidance on memory safety vulnerabilities. 2024. URL: <https://www.cisa.gov/news-events/news/urgent-need-memory-safety-software-products>.
- [Dus+22] Edlira Dushku et al. “Towards Heterogeneous Remote Attestation Protocols”. In: *Proceedings of the 17th International Conference on Security and Cryptography*. SCITEPRESS, 2022, pp. 112–123. DOI: 10.5220/0011289000003283.
- [Ede25] Edera. *Remote Attestation in Confidential Computing Explained*. Edera blog. Accessed: August 11, 2025. July 2025. URL: <https://edera.dev/stories/remote-attestation-in-confidential-computing-explained>.
- [Int25] Internet Assigned Numbers Authority. *Transport Layer Security (TLS) ExtensionType Values*. IANA Registry. Accessed: August 11, 2025. 2025. URL: <https://www.iana.org/assignments/tls-extensiontype-values/tls-extensiontype-values.xhtml>.
- [Par24] Parallax Second. *TSS 2.0 Enhanced System API (ESAPI) Rust wrapper*. Rust wrapper for TCG TSS 2.0 Enhanced System API. 2024. URL: <https://docs.rs/tss-esapi/>.

- [Res18] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. Standards Track. Internet Engineering Task Force, Aug. 2018. URL: <https://tools.ietf.org/rfc/rfc8446.txt>.
- [Rus24] Rustls Contributors. *Rustls: A Modern TLS Library in Rust*. API documentation and library overview. Rustls Project. 2024. URL: <https://docs.rs/rustls/latest/rustls/>.
- [Sal+08] Joseph Salowey, Hao Zhou, Pasi Eronen, and Hannes Tschofenig. *Transport Layer Security (TLS) Session Resumption without Server-Side State*. RFC 5077. Stateless TLS session resumption mechanisms. Internet Engineering Task Force, Jan. 2008. URL: <https://tools.ietf.org/rfc/rfc5077.txt>.
- [The24] The Rust Team. *The Rust Programming Language: Understanding Ownership*. Accessed: August 11, 2025. Rust Foundation. 2024. URL: <https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html>.
- [Tru19] Trusted Computing Group. *TPM 2.0 Library Specification*. Family 2.0, Level 00, Revision 01.59. 2019. URL: <https://trustedcomputinggroup.org/resource/tpm-library-specification/>.
- [Uti24] Utimaco. *What is Root of Trust?* Utimaco Knowledge Base. Accessed: August 11, 2025. 2024. URL: <https://utimaco.com/service/knowledge-base/hardware-security-modules/what-root-trust>.
- [Wei+25] Carsten Weinhold, Muhammad Usama Sardar, Ionut Mihalcea, Yogesh Deshpande, Hannes Tschofenig, Yaron Sheffer, Thomas Fossati, and Michael Roitzsch. “Separate but Together: Integrating Remote Attestation into TLS”. In: *Proceedings of the 2025 USENIX Annual Technical Conference (ATC ’25)*. Boston, MA, USA: USENIX Association, July 2025, pp. 1319–1326. ISBN: 978-1-939133-48-9.
- [WWR22] Robert Walther, Carsten Weinhold, and Michael Roitzsch. “RATLS: Integrating Transport Layer Security with Remote Attestation”. In: *4th Workshop on Cloud Security and Privacy (Cloud S&P)*. Rome, Italy: Springer International Publishing, June 2022, pp. 361–379. DOI: 10.1007/978-3-031-16815-4_20.
- [YH24] Xue Yang and Enda Howley. “ADT: Time series anomaly detection for cyber-physical systems via deep reinforcement learning”. In: *Computers & Security* 139 (2024), p. 103711. DOI: 10.1016/j.cose.2024.103711.